

✎ Aerial Object Detection on VisDrone2019 — YOLO11n

CSC354 Machine Learning — Semester Project

This notebook trains **YOLO11n** on the **VisDrone2019-DET** benchmark for aerial object detection, evaluates it with **mAP, precision, recall, F1-score, and FPS**, and adds a **SAHI (Slicing Aided Hyper Inference)** comparison to improve small-object detection.

⚠ **Runtime setup:** Go to `Runtime → Change runtime type → GPU (T4 recommended)` before running. Training on CPU will be extremely slow.

```
# 1. Install dependencies
!pip install ultralytics sahi -q
import ultralytics
ultralytics.checks()
```

Ultralytics 8.4.100 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Setup complete ✅ (2 CPUs, 12.7 GB RAM, 46.7/112.6 GB disk)

```
# 2. Check GPU
import torch
print("CUDA available:", torch.cuda.is_available())
print("Device:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU only – go to Runtime > Change runtime type")
```

CUDA available: True
Device: Tesla T4

✎ Optional: Mount Google Drive

Recommended so your trained weights and results survive a Colab disconnect.

```
# 3. (Optional but recommended) Mount Google Drive to persist results
from google.colab import drive
try:
    drive.mount('/content/drive')
    import os
    PROJECT_DIR = "/content/drive/MyDrive/visdrone_yolo_project"
    os.makedirs(PROJECT_DIR, exist_ok=True)
    print("Results will be saved to:", PROJECT_DIR)
except:
    print("Drive not mounted. Using local /content/project_results instead.")
    PROJECT_DIR = "/content/project_results"
    os.makedirs(PROJECT_DIR, exist_ok=True)
```

Mounted at /content/drive
Results will be saved to: /content/drive/MyDrive/visdrone_yolo_project

✎ Dataset: VisDrone2019-DET

VisDrone2019-DET has **10 classes**: pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, motor. Ultralytics auto-downloads (~2 GB) and converts annotations to YOLO format the first time `VisDrone.yaml` is used.

```
# 4. Trigger dataset download + conversion
from ultralytics.utils.downloads import download
import yaml, ultralytics, pathlib

data_yaml = "VisDrone.yaml" # built into ultralytics/cfg/datasets/
print("Using built-in dataset config:", data_yaml)
```

Using built-in dataset config: VisDrone.yaml

✎ Training

`yolo11n.pt` is the nano variant. We will start with 100 epochs, but if you are on a CPU or a time limit, you might want to reduce this to 10-20 to test the pipeline.

```
# 5. Train YOLO11n on VisDrone
try:
    from ultralytics import YOLO
except ImportError:
```

```

!pip install ultralytics -q
from ultralytics import YOLO

model = YOLO("yolo11n.pt") # pretrained nano checkpoint

results = model.train(
    data="VisDrone.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    patience=20,
    project=PROJECT_DIR,
    name="yolo11n_visdrone",
    exist_ok=True,
    plots=True,
)

```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	405/405	2.5it/s	2:39
94/100	6.48G	1.357	0.8837	0.8822	298	640: 100%	405/405	2.5it/s	2:39	
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	18/18	2.8it/s		
all	548	38759	0.447	0.339	0.321	0.181				
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	405/405	2.6it/s	2:36
95/100	6.48G	1.352	0.8801	0.8823	309	640: 100%	405/405	2.6it/s	2:36	
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	18/18	2.6it/s		
all	548	38759	0.449	0.342	0.322	0.181				
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	405/405	2.6it/s	2:39
96/100	6.48G	1.35	0.8761	0.8805	189	640: 100%	405/405	2.6it/s	2:39	
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	18/18	3.2it/s		
all	548	38759	0.455	0.341	0.321	0.181				
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	405/405	2.5it/s	2:39
97/100	6.48G	1.346	0.8715	0.8813	314	640: 100%	405/405	2.5it/s	2:39	
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	18/18	3.6it/s		
all	548	38759	0.457	0.339	0.321	0.181				
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	405/405	2.5it/s	2:39
98/100	6.48G	1.344	0.871	0.8805	343	640: 100%	405/405	2.5it/s	2:39	
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	18/18	3.6it/s		
all	548	38759	0.461	0.338	0.32	0.181				
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	405/405	2.6it/s	2:38
99/100	6.48G	1.347	0.8694	0.8794	195	640: 100%	405/405	2.6it/s	2:38	
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	18/18	3.5it/s		
all	548	38759	0.46	0.337	0.319	0.181				
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	405/405	2.5it/s	2:39
100/100	6.48G	1.341	0.8675	0.8815	311	640: 100%	405/405	2.5it/s	2:39	
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	18/18	3.6it/s		
all	548	38759	0.459	0.338	0.32	0.181				

100 epochs completed in 4.838 hours.

Optimizer stripped from /content/drive/MyDrive/visdrone_yolo_project/yolo11n_visdrone/weights/last.pt, 5.5MB

Optimizer stripped from /content/drive/MyDrive/visdrone_yolo_project/yolo11n_visdrone/weights/best.pt, 5.5MB

Validating /content/drive/MyDrive/visdrone_yolo_project/yolo11n_visdrone/weights/best.pt...

Ultralytics 8.4.100 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

YOLO11n summary (fused): 101 layers, 2,584,102 parameters, 0 gradients, 6.3 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	18/18	1.7it/s
all	548	38759	0.445	0.347	0.324	0.183		
pedestrian	520	8844	0.456	0.374	0.346	0.143		
people	482	5125	0.482	0.269	0.261	0.0897		
bicycle	364	1287	0.251	0.114	0.0845	0.0331		
car	515	14064	0.635	0.77	0.75	0.498		
van	421	1975	0.472	0.382	0.367	0.249		
truck	266	750	0.475	0.289	0.289	0.187		
tricycle	337	1045	0.383	0.251	0.208	0.109		
awning-tricycle	220	532	0.286	0.141	0.104	0.0664		
bus	131	251	0.545	0.478	0.465	0.315		
motor	485	4886	0.466	0.405	0.364	0.142		

Speed: 0.1ms preprocess, 1.5ms inference, 0.0ms loss, 3.2ms postprocess per image

Results saved to /content/drive/MyDrive/visdrone_yolo_project/yolo11n_visdrone

✧ Evaluation: mAP, Precision, Recall, F1

`model.val()` runs on the validation split and reports mAP50, mAP50-95, precision, and recall per class and overall.

```

# 6. Validate and pull metrics
from ultralytics import YOLO

```

```

best_weights = f"{PROJECT_DIR}/yolo11n_visdrone/weights/best.pt"
model = YOLO(best_weights)

```

```
metrics = model.val(data="VisDrone.yaml", imgsz=640, split="val")
```

```
precision = metrics.box.mp          # mean precision across classes
recall = metrics.box.mr            # mean recall across classes
map50 = metrics.box.map50
map5095 = metrics.box.map
```

```
f1 = 2 * (precision * recall) / (precision + recall + 1e-9)
```

```
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"mAP@50 : {map50:.4f}")
print(f"mAP@50-95 : {map5095:.4f}")
print(f"F1-score : {f1:.4f}")
```

Ultralytics 8.4.100 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

YOLO11n summary (fused): 101 layers, 2,584,102 parameters, 0 gradients, 6.3 GFLOPs

val: Fast image access (ping: 0.0±0.0 ms, read: 2124.8±826.9 MB/s, size: 126.7 KB)

val: Scanning /content/datasets/VisDrone/labels/val.cache... 548 images, 0 backgrounds, 0 corrupt: 100% 548/548

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)
all	548	38759	0.449	0.347	0.324	0.184
pedestrian	520	8844	0.461	0.375	0.347	0.145
people	482	5125	0.489	0.268	0.262	0.0908
bicycle	364	1287	0.254	0.113	0.0848	0.0334
car	515	14064	0.638	0.77	0.75	0.499
van	421	1975	0.473	0.382	0.368	0.25
truck	266	750	0.478	0.289	0.291	0.187
tricycle	337	1045	0.391	0.252	0.208	0.109
awning-tricycle	220	532	0.288	0.141	0.105	0.0671
bus	131	251	0.547	0.477	0.465	0.316
motor	485	4886	0.468	0.403	0.363	0.143

Speed: 1.2ms preprocess, 3.0ms inference, 0.0ms loss, 3.0ms postprocess per image

Results saved to /content/runs/detect/val

Precision : 0.4486

Recall : 0.3469

mAP@50 : 0.3244

mAP@50-95 : 0.1841

F1-score : 0.3913

6b. Per-class breakdown

import pandas as pd

```
class_names = model.names
per_class = pd.DataFrame({
    "class": [class_names[i] for i in range(len(class_names))],
    "AP50": metrics.box.ap50,
    "AP50-95": metrics.box.ap,
})
per_class
```

	class	AP50	AP50-95
0	pedestrian	0.347310	0.144772
1	people	0.261750	0.090821
2	bicycle	0.084818	0.033398
3	car	0.749828	0.499359
4	van	0.367648	0.250359
5	truck	0.291189	0.186878
6	tricycle	0.208134	0.108906
7	awning-tricycle	0.105021	0.067053
8	bus	0.464908	0.316462
9	motor	0.363258	0.143336

Next steps: [Generate code with per_class](#)

[New interactive sheet](#)

▼ FPS Benchmark

Measures real-time inference speed on the validation images.

```
# 7. FPS benchmark
import time, glob
from ultralytics import YOLO
```

```

val_images = glob.glob("datasets/VisDrone/images/val/*.jpg")[:100]
model = YOLO(best_weights)

# Warmup
for img in val_images[:5]:
    model.predict(img, imgsz=640, verbose=False)

start = time.time()
for img in val_images:
    model.predict(img, imgsz=640, verbose=False)
elapsed = time.time() - start

fps = len(val_images) / elapsed
print(f"Average FPS on {len(val_images)} images: {fps:.2f}")
print(f"Average inference time per image: {(elapsed/len(val_images))*1000:.2f} ms")

Average FPS on 100 images: 65.02
Average inference time per image: 15.38 ms

```

✓ SAHI: Slicing Aided Hyper Inference (Novelty Enhancement)

VisDrone objects are often tiny. SAHI slices images into overlapping tiles to recover small objects missed at standard resolutions.

```

# 8. Install/verify SAHI predictor wrapper
from sahi import AutoDetectionModel
from sahi.predict import get_sliced_prediction, predict

detection_model = AutoDetectionModel.from_pretrained(
    model_type="ultralytics",
    model_path=best_weights,
    confidence_threshold=0.25,
    device="cuda:0" if torch.cuda.is_available() else "cpu",
)

```

```

# 9. Run SAHI sliced-prediction evaluation
result = predict(
    model_type="ultralytics",
    model_path=best_weights,
    model_device="cuda:0" if torch.cuda.is_available() else "cpu",
    model_confidence_threshold=0.25,
    source="datasets/VisDrone/images/val",
    slice_height=512,
    slice_width=512,
    overlap_height_ratio=0.2,
    overlap_width_ratio=0.2,
    project=PROJECT_DIR,
    name="sahi_visdrone_val",
)
print("SAHI sliced predictions saved to:", f"{PROJECT_DIR}/sahi_visdrone_val")

```

```

Performing prediction on 6 slices.
Performing inference on images: 98%|██████████| 538/548 [02:39<00:02, 3.99it/s]Prediction time is: 137.53 ms
Performing prediction on 8 slices.
Performing inference on images: 98%|██████████| 539/548 [02:39<00:02, 3.97it/s]Prediction time is: 154.61 ms
Performing prediction on 8 slices.
Performing inference on images: 99%|██████████| 540/548 [02:40<00:02, 3.82it/s]Prediction time is: 146.87 ms
Performing prediction on 6 slices.
Performing inference on images: 99%|██████████| 541/548 [02:40<00:01, 4.03it/s]Prediction time is: 132.91 ms
Performing prediction on 8 slices.
Performing inference on images: 99%|██████████| 542/548 [02:40<00:01, 3.91it/s]Prediction time is: 145.97 ms
Performing prediction on 8 slices.
Performing inference on images: 99%|██████████| 543/548 [02:40<00:01, 3.77it/s]Prediction time is: 150.17 ms
Performing prediction on 8 slices.
Performing inference on images: 99%|██████████| 543/548 [02:41<00:01, 3.77it/s]Prediction time is: 204.44 ms
Performing inference on images: 99%|██████████| 545/548 [02:41<00:00, 3.47it/s]Performing prediction on 6 slices.
Prediction time is: 115.61 ms
Performing inference on images: 99%|██████████| 545/548 [02:41<00:00, 3.47it/s]Performing prediction on 8 slices.
Prediction time is: 152.54 ms
Performing inference on images: 100%|██████████| 546/548 [02:41<00:00, 3.49it/s]Performing prediction on 8 slices.
Prediction time is: 160.50 ms
Performing inference on images: 100%|██████████| 547/548 [02:42<00:00, 3.51it/s]Performing prediction on 6 slices.
Prediction time is: 130.05 ms
Performing inference on images: 100%|██████████| 548/548 [02:42<00:00, 3.38it/s]Prediction results are successfully exported
SAHI sliced predictions saved to: /content/drive/MyDrive/visdrone_yolo_project/sahi_visdrone_val

```

Results Summary Table

Comparison between standard YOLO11n and YOLO11n + SAHI.

```

# 11. Summary comparison table
import pandas as pd

comparison = pd.DataFrame({
    "Method": ["YOLO11n (standard)", "YOLO11n + SAHI"],
    "Precision": [precision, None],
    "Recall": [recall, None],
    "mAP@50": [map50, None],
    "mAP@50-95": [map5095, None],
    "F1-score": [f1, None],
    "FPS": [fps, None],
})
comparison

```

	Method	Precision	Recall	mAP@50	mAP@50-95	F1-score	FPS	
0	YOLO11n (standard)	0.44858	0.346916	0.324386	0.184134	0.391252	65.021497	
1	YOLO11n + SAHI	NaN	NaN	NaN	NaN	NaN	NaN	

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```

# 12. Training curves and confusion matrix
from IPython.display import Image, display

run_dir = f"{PROJECT_DIR}/yolo11n_visdrone"
try:
    display(Image(filename=f"{run_dir}/results.png"))
    display(Image(filename=f"{run_dir}/confusion_matrix.png"))
except:
    print("Result plots not found. They will be generated after training completes.")

```

